

Hero Radar：更早发现下一个 AI 应用层爆发项目

1. 简介

题目是：更早监控到下一个 Hermes 或 OpenClaw。

我的解法是每天从 GitHub、HN、Product Hunt、npm、PyPI、Hugging Face、X 等公开来源抓信号，把同一个项目的跨来源证据合 agent LLM 做打分，生成 Daily Feed。

目标是找到还没有完全成为共识、但已经出现产品或工作流突破的项目，更早捕捉产品先机。

这个系统关注三类信号并筛选值得关注的产品：

- 增长信号：stars、forks、downloads、rank、mentions 的变化速度，通过 deterministic rules 做第一层筛选。
- 交叉信号：同一个项目是否在多个 source 里同时出现，通过 entity grouping 和 classifier 合并证据。
- 产品信号：项目是否真的改变了一个使用场景或工作流，通过 agent LLM 做打分和分析。

最终用户每天看到的结果：

- 今天最值得看的项目。
- 它为什么值得看。
- 支撑证据来自哪里。

2. 产品前期调查

2.1 市场分析

公开市场里已经有很多原始信号。

- 项目发布平台：GitHub、Product Hunt。
- 科技讨论平台：HN、X。
- 包和模型平台：npm、PyPI、Hugging Face。

也有一些工具已经在做趋势判断，比如 RepoFOMO、Trending Repos、Trendshift。

但目前大部分方案仍然有三个限制：

- Source 偏单一：趋势判断主要基于单个平台，例如 GitHub 增长、HN 讨论或 Product Hunt launch，缺少跨平台项目聚合。
- 判断偏 deterministic：主要基于 stars、rank、upvotes、downloads、velocity 等指标，缺少对项目本身的 qualitative analysis。
- 加速度权重不够：很多工具更关注当前热度或排名，对项目爆发前的 acceleration 关注不够。

Hero Radar 选择从多源 evidence 入手，再用 LLM 做 qualitative judgment。核心价值在于把分散信号变成产品判断。

2.2 回溯研究

市场上有很多信号，但产品问题还是要回到一句话：

什么项目算好项目？

为了把这个问题具体化，我用题目里的 Hermes 和 OpenClaw 做回溯。方法是收集这两个项目爆发前后的多渠道历史数据，观察它 source 发生了什么。

回溯方法

- 选取 Hermes 和 OpenClaw 作为已知爆发项目。
- 收集项目爆发前后的 GitHub、HN、Product Hunt、npm、Hugging Face 等历史数据。
- 对比信号出现时间、增长加速度、跨 source 共振、alias 迁移。
- 用当时可见的数据反推：系统能否在项目成为共识前把它放进候选池。

回溯数据 OpenClaw 的外部共识窗口我用 2026-01-29 到 2026-01-30 作为锚点：2026-01-29 [Axios](#) 已经把 Moltbot/Clawdbot 作为硅谷新一轮 AI agent 关注对象报道；2026-01-30 [外部报道](#)开始出现 viral 和 100k stars 叙述。

项目	早期信号	关键数字	发现
Hermes	GitHub acceleration 先出现，HN 有弱 corroboration	acceleration T0: 2026-03-11; star band 中位约 4.45k; 后续 velocity peak: 2026-04-06; star band 中位约 30.5k	如果在 acceleration 阶段入池，可以比后续共识阶段早约 26 天看到
OpenClaw	旧 alias clawdbot 先在 npm、HN、Hugging Face 出现，外部共识窗口在 2026-01-29 到 2026-01-30	npm downloads: 2026-01-25 为 70,768, 2026-01-26 为 106,024; @clawdbot/* 4 个 scoped packages 在 2026-01-25 同步发布; HN 旧 alias 在 2026-01-25 到 2026-01-27 约 42 条 stories	系统可以在 2026-01-25 通过 npm floor + package burst + HN 旧 alias 入池，比外部共识窗口早约 4-5 天

回溯 implication

- Candidate rules 要关注 acceleration。total popularity 适合作为结果确认，acceleration 更适合早期入池。
- Entity grouping 要支持 repo、domain、package、alias、source link、canonical URL 的合并。OpenClaw 这种 case 不能只靠项目名。
- Cross-source evidence 要进入候选逻辑。多个弱信号在短时间内同时出现，比单个 source 的绝对热度更有参考价值。
- LLM scorer 要判断项目本身的产品含义。deterministic rules 能发现变化，LLM 负责判断这个变化是否代表产品或 workflow 突破。

3. 产品考量和解法

3.1 HN 和 X 的信号早，但噪音高

HN 和 X 往往能最早看到项目被讨论。

但里面混着文章、教程、泛概念、公司新闻和无链接推荐。

解法：

- 先用 LLM 做 source cleaner，只判断这条信号是否指向具体项目，并提取项目名、链接、summary 和 confidence。
- X 再做 entity-level 聚合，看是否有明确 citation、多个可信作者、以及和同一个项目的稳定绑定；模糊匹配和无 citation 的信号降级或丢弃。

3.2 LLM 打分慢且贵，需要分层使用

每天抓到的 source item 很多。

全量 LLM 打分会带来成本、延迟和噪音问题。

解法：

- 第一层用 deterministic rules 和 entity grouping 做高召回候选池，把几千条 raw signal 压到少量候选项目。
- 第二层再使用 LLM：classifier 用 DeepSeek 这类更便宜的模型清理 noisy source，scorer 用 Kimi 这类更强的模型对候选项目做 agentic investigation 和 Daily Feed brief。

4. 产品视图和用户路径

4.1 User journey

Hero Radar 的用户路径从判断开始，再逐层下钻到证据和原始数据。

1. 用户每天先打开 Daily Feed，看 Kimi scorer 挑出来的重点项目。
2. 如果想看 LLM 还没打分、只靠 data/rules 筛出来的当日候选，进入 Candidate Pool。
3. 如果想看今天抓到的全量项目，或验证某个判断，回到 Sources 看原始 evidence。
4. 如果想调整雷达的范围，进入 Settings 修改 source、关键词、模型和预算。

这四个页面对应四种不同层级的注意力分配：

- Daily Feed: 看 Kimi scorer picked items, 直接消费判断。
- Candidate Pool: 看当日 data picked items, 保留更宽的早期观察面。
- Sources: 看全量 source-native data, 用来浏览和验证。
- Settings: 控制雷达看哪里、花多少预算、重点追踪什么。

4.2 产品视图和功能

a. Daily Feed Daily Feed 是每天真正消费的页面。

它把大量 source noise 压缩成少量 Kimi scorer picked items。

用户能看到:

- 今日重点项目。
- 候选信号。
- 完整评分记录。
- 项目分数和关键维度。
- 支撑证据和 source links。
- brief、use case、caveat。

Kimi scorer 主要按四个产品维度打分:

- Workflow shift: 项目是否改变了某个具体工作流。
- Technical substance: 项目是否有足够技术或产品实质。
- Product-market fit: 是否有清晰用户、场景和采用路径。
- Momentum: 当前增长和跨源证据是否可信。

这里有两个层面的 brief:

- 评分层 brief: 所有被 Kimi scorer 看过的项目都会有结构化分数和简短判断, 方便用户快速扫。
- 重点层 brief: 进入今日重点的项目会有更完整的中文分析, 解释它改变的工作流、适合关注的 use case、以及主要 caveat。

Feed 里也需要处理权重问题。

例如 OpenAI、Anthropic、Google、Microsoft 这类大厂项目可能很重要, 但它们容易长期占据注意力。所以这类项目可以被打分和留档, 同时标记为大厂或 score-only, 避免挤占早期项目的今日重点位置。

用户能做的操作:

- 快速判断今天是否有值得深入看的项目。
- 打开项目链接或 source evidence。
- 查看项目为什么被打分。
- 对 Feed 结果给反馈, 后续沉淀成个人偏好。

b. Candidate Pool Candidate Pool 是高召回候选池。

它保存还没有进入重点推荐、但已经出现足够信号的项目。

这个视图回答的问题是:

为什么这个项目值得进入观察?

用户能看到:

- 项目来自哪些 source。
- 哪些 signal 让它入池。
- 是 acceleration、rank、download burst、HN 讨论、X mention, 还是跨源共振。
- classifier 发现的 alias、link、canonical name。
- 当前 level: high potential、potential、edge watch。
- 项目的 context preview 和原始链接。

Candidate Pool 的价值是保留早期不确定性。

有些项目还没有足够证据进入 Daily Feed, 但已经不应该被丢掉。

Candidate Pool 给用户一个更宽的观察面。

用户能做的操作:

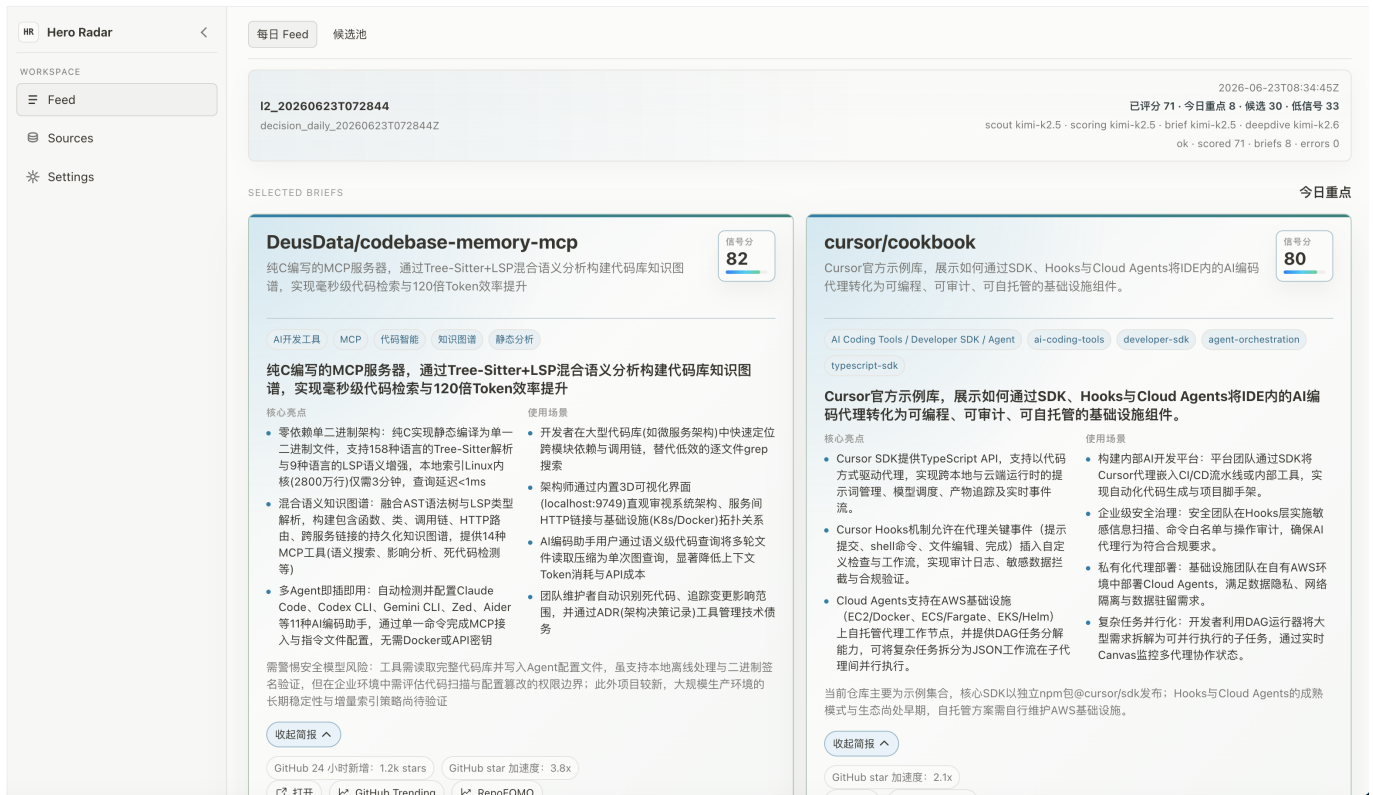


Figure 1: Daily Feed

- 按 source 或 level 筛选候选。
- 查看候选项目的入池原因。
- 对比同一个项目在多个 source 里的证据。
- 从候选池继续打开项目主页、repo 或原始讨论。

c. Sources Sources 是原始事实层。它的作用是让判断可追溯。

用户能看到不同 source 的 source-native facts:

- GitHub repo 的 stars、forks、velocity、rank。
- HN 的 title、points、comments、time。
- Product Hunt 的 rank、votes、comments。
- npm / PyPI 的 package、downloads、release 信息。
- Hugging Face 的 models、spaces、datasets。
- X 的 tweets、authors、mentions。

Sources 适合两种场景:

- 用户想验证 Feed 或 Candidate Pool 的判断。
- 用户想直接浏览某个 source 今天发生了什么。

这里尽量保留原始数据形态。模型判断放在 Feed 和 Candidate Pool, Sources 负责提供证据底座。

d. Settings Settings 是定义雷达边界的地方。

用户能控制:

- 哪些 source 开启。
- 每个 source 抓多少。
- GitHub / HN / npm / X 搜什么关键词。

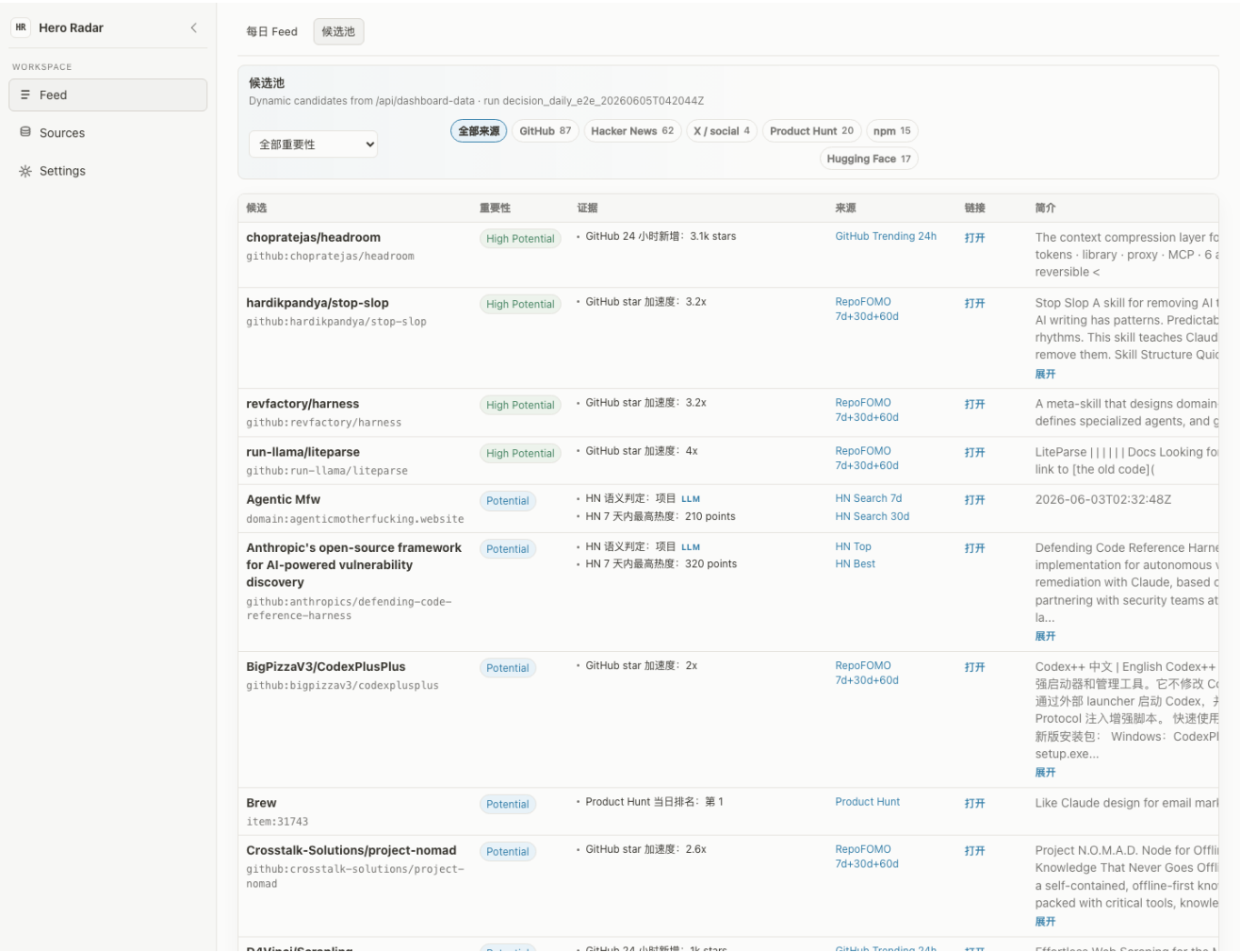


Figure 2: Candidate Pool

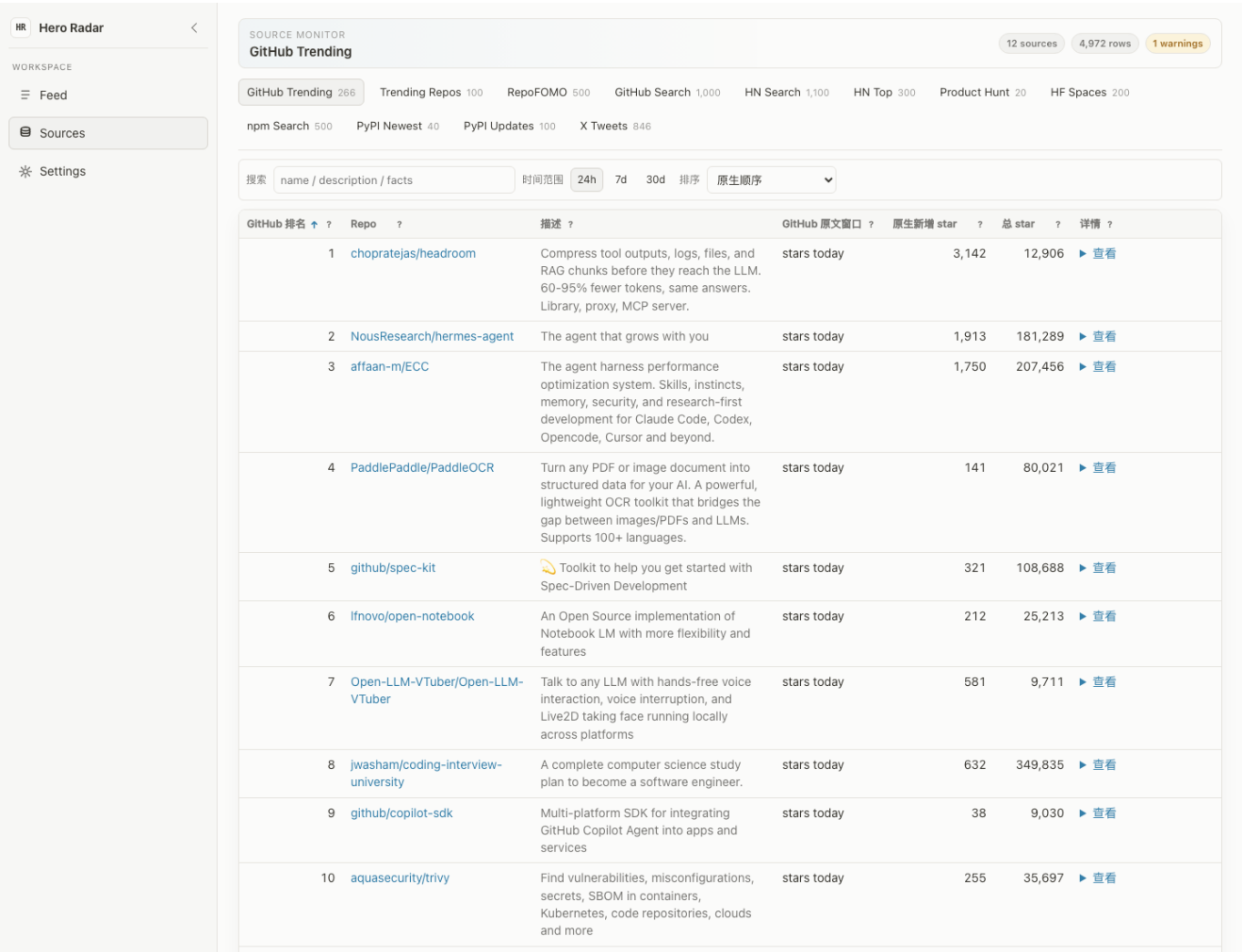


Figure 3: Sources

- X 追踪哪些账号或 seed network。
- Kimi scorer 使用哪个模型。
- Kimi scorer 能花多少预算。
- 是否允许 web search。
- API key 和 source 状态。

Settings 的产品意义是透明和可控。

它告诉用户：今天这个雷达看了哪里，没看哪里，花了多少模型预算，判断建立在哪些 source 上。

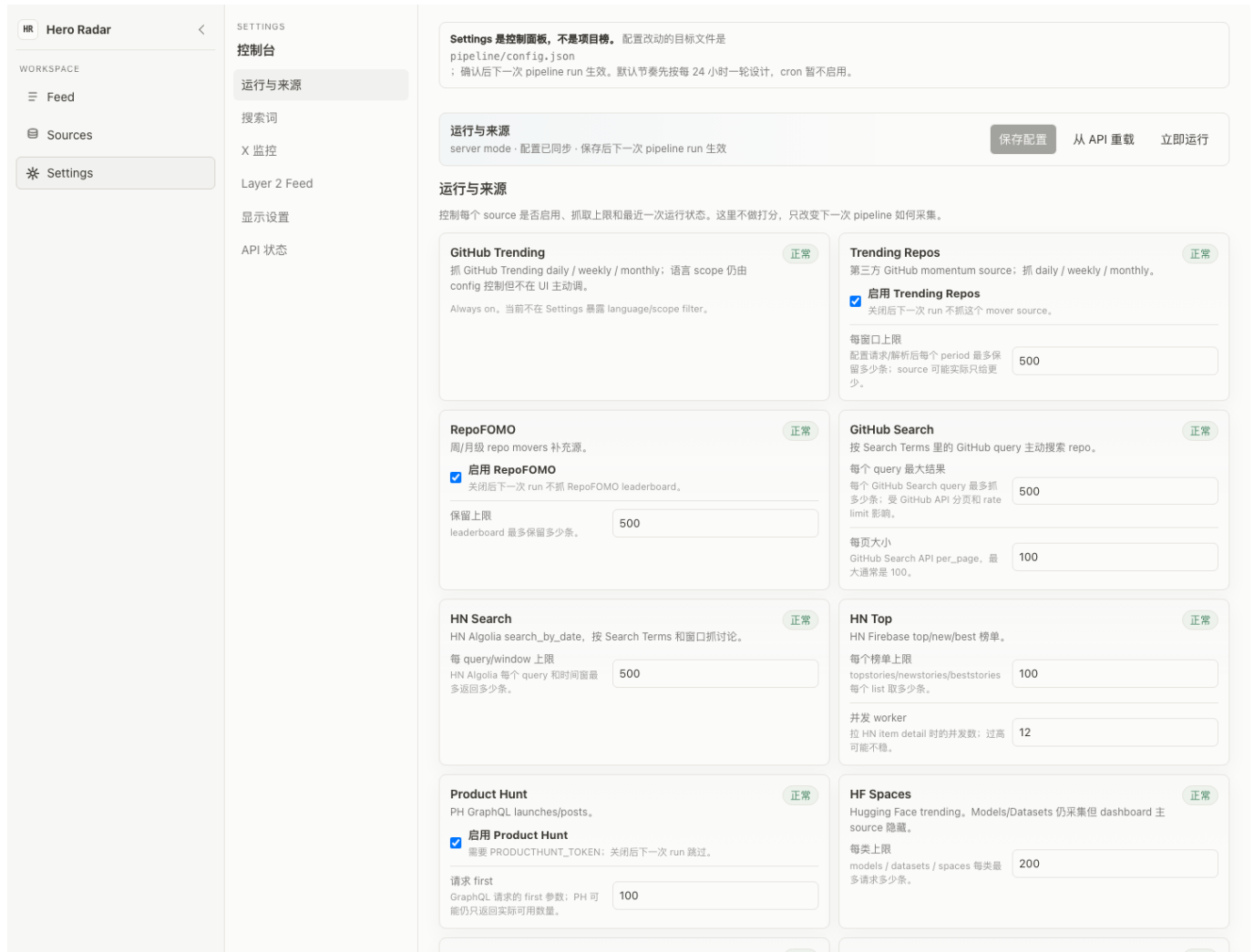


Figure 4: Settings

5. 项目亮点

- 多源覆盖：同时看 GitHub、HN、Product Hunt、npm、PyPI、Hugging Face、X，覆盖 repo 增长、社区讨论、产品发布、包发布和社交传播。
- 提前发现：Hermes 可在 2026-03-11 acceleration 阶段入池，早于 2026-04-06 velocity peak 约 26 天；OpenClaw 可在 2026-01-25 通过 clawdbot npm/HN 信号入池，早于 2026-01-29 到 2026-01-30 外部共识窗口约 4-5 天。
- 跨源聚合：同一个项目可能用 repo、package、domain、alias、source link 多种形态出现，系统会把这些证据合并到同一个候选。
- 可溯源：Feed 和 Candidate Pool 里的判断都能回到原始 repo、HN story、Product Hunt launch、npm package、HF resource、X tweet。
- 可调控：用户可以调整 source 开关、抓取量、GitHub/HN/npm/X 关键词、X/Twitter 账号、Kimi scorer 预算和 web search 权限。

6. 技术实现

6.1 数据采集层

Pipeline 每天从多个 source 拉取数据，并统一成 source item。

主要 source 包括：

- GitHub Trending / Search。
- Trending Repos / RepoFOMO。
- HN Algolia / Firebase。
- Product Hunt。
- Hugging Face。
- npm / PyPI。
- X seed accounts / tweets。

每条 source item 保留 source、external id、name、url、heat、velocity seed、rank、description、metadata 和 raw data。

6.2 Entity resolution / grouping

系统把不同 source 里的同一项目合并成 entity。

优先使用确定性 key：

- GitHub repo。
- domain。
- package link。
- canonical URL。
- source metadata。

名字相似只作为弱线索。

这个阶段解决的问题是：同一个项目可能在不同 source 里以不同名字出现。

6.3 Candidate rules

Candidate rules 根据 evidence 选择候选。

规则包括：

- GitHub acceleration / velocity。
- Trending Repos / RepoFOMO 增长。
- HN points / story count。
- Product Hunt rank。
- Hugging Face resources。
- npm download burst / package family。
- X social evidence。
- 48 小时内多个弱信号共振。

候选分为 high potential、potential 和 edge watch。

6.4 Classifier pipeline

Classifier pipeline 处理 noisy source。

HN classifier 判断 projectness，并提取 canonical name、links、summary 和 confidence。

X classifier 分两阶段：

1. tweet-level triage。
2. entity-level aggregation。

Classifier 输出 evidence rows、alias links 和 merge proposals，供 grouping 和 candidate selection 使用。

7. Kimi scorer mechanism

Kimi scorer 是一个 agentic workflow，用 ReAct-style loop 对候选项目做二次研究和评分。

它先读候选上下文，判断是否需要补证据；如果需要，就调用工具；工具结果回来后继续判断；证据足够或预算耗尽后输出 final score 和 brief。

7.1 ReAct-style scoring loop

Kimi scorer 的每个候选项目独立运行。

基本流程：

1. 输入候选项目的 compact context，包括项目名、source links、candidate level、evidence summary 和已有 meta-data。
2. Kimi 判断当前上下文是否足够打分。
3. 如果不够，输出 use_tools，请求最小必要工具。
4. 系统执行工具，把结果压缩后放回上下文。
5. Kimi 继续下一轮判断。
6. 最终输出 final，包含 score、supporting evidence、negative evidence、known gaps、should print 和 brief。

当前预算设计：

- 最多 3 个 investigation turns。
- 每个候选最多 8 次 tool calls。
- web search、repo file、homepage/docs 都有单独上限。
- 候选之间独立执行，单个失败不会影响整次 feed run。

7.2 Scoring rubric

Kimi scorer 主要看四个产品维度：

- workflow shift。
- technical substance。
- product-market fit。
- momentum。

这样做的原因是：项目早期热度本身不够，模型需要判断热度背后的产品含义。

- workflow shift 负责判断工作流变化。
- technical substance 负责过滤空壳、普通 wrapper 和纯发布新闻。
- product-market fit 负责判断用户、场景和采用路径。
- momentum 负责结合增长和跨源证据。

实现上还有一组硬规则和 route 规则：

- 新闻、教程、纯模型发布、资源列表会被压低。
- 大厂常规动态可以 score-only，不挤占今日重点。
- 证据不足时要求模型降低确定性，并写出 known gaps。

7.3 Tool design 和 guardrails

Kimi scorer 能用的工具很少，都是 primitive tools：

- read_evidence_rows：读取结构化 evidence，避免模型只看摘要。
- fetch_github_readme：读取 repo README，判断项目定位和使用方式。
- fetch_github_file：读取少量白名单 repo 文件，例如 package.json、pyproject.toml、README、docs/readme.md。
- fetch_homepage_or_docs：读取官网或文档页面。
- web_search：可选工具，只在需要外部补证据时使用。

工具设计的原则是让模型补关键证据，并限制开放式浏览。

Guardrails 包括：

- URL 安全校验，避免访问 localhost、private IP、reserved IP。
- GitHub file path 白名单，避免任意读 repo。

- tool call 次数上限。
- investigation turn 上限。
- web search 次数上限。
- 输出 JSON schema 校验。
- schema repair。

7.4 Context management 和 compaction

Kimi scorer 不能直接吃下所有 raw data。

这里的 compaction 采用结构化压缩和按需取证，避免把大量 raw content 交给模型自由总结。

第一层是 candidate context。

- Grouping 先把同一个项目的 GitHub、HN、npm、HF、X evidence 合成一个候选。
- 进入 Kimi scorer 的上下文只保留项目级信息：canonical name、candidate level、source links、evidence summary、metadata。
- unrelated source rows 不进入 prompt。

第二层是 evidence budget。

- read_evidence_rows 有条数上限，默认最多读取 80 条 evidence。
- README、repo file、homepage/docs 都有字符上限，默认单次 tool result 截到约 6000 chars。
- web search 默认每个候选最多 1 次，避免模型用搜索结果覆盖已有证据。

第三层是 tool result compaction。

- 工具返回的内容只保留判断需要的字段，例如 title、url、snippet、README 摘要、package metadata。
- 每次 tool call 的结果会写入 tool trace。
- 下一轮 prompt 只带必要 observation，不把完整数据库和完整 trace 全部塞回去。

第四层是 trace separation。

- SQLite 保存完整 source items、evidence rows、turn trace、tool trace 和最终 score。
- Kimi scorer 只看到当前判断需要的 compact context。
- 用户或开发者后续可以回看完整 trace，复盘为什么这个项目被打分。

这个设计的目标是保留足够信息密度，同时防止 context 被 raw data 淹没。

7.5 Eval

Eval case 包括：

- 高分样本：OpenClaw、Hermes、HeyClicky。
- 低分样本：funding news、普通教程、模型发布、generic chatbot、普通 dashboard。
- 中间样本：screen-aware spreadsheet operator。

Eval 的作用是校准产品判断。

系统需要知道什么应该被推荐，也要知道什么应该被压低。

8. 未来方向

8.1 MCP as intelligence layer

Hero Radar 更适合先作为信息聚合层开放出去。

产品主入口继续保持 dashboard。

这个产品的主要消费场景是每天看 brief、看候选、看证据，用 chat box 作为主入口价值不高。

未来更合理的方向是 MCP。

- 查询今天的高分项目。
- 读取某个项目的 evidence rows。
- 请求 Kimi scorer brief。
- 把项目加入 watchlist。
- 修改 source、关键词、预算、X/Twitter 账号等设定。

- Explore 本地 database, 让用户自己的 agent 做后续 research。

MCP 更适合这个场景。

- 用户自己的 agent 需要 dedicated tools。
- Evidence、candidate、feed、settings 都可以做成稳定 tool surface。
- Dashboard 负责人看, MCP 负责 agent 用。

8.2 Feedback-driven self-evolving harness

未来用户看到打分和 brief 后, 可以直接反馈。

交互可以很简单:

- thumbs up / thumbs down。
- 一个 reason box, 写为什么这个项目好或不好。

当前 harness 还没有 memory 模块。

未来需要新增一个 memory layer, 把用户反馈沉淀成可调用的偏好和 eval 数据。

这些反馈进入 self-evolving harness:

- 好项目和坏项目会沉淀成新的 eval case。
- 用户的理由会被转成 preference notes。
- Memory module 记录用户长期偏好, 例如更关注 devtools、prosumer workflow、enterprise adoption, 或更反感普通 wrapper。
- Eval case 会反过来校准 Kimi scorer 的 rubric、权重和打分规则。
- 高频负反馈会影响 source 权重、candidate ranking 和 brief 关注点。

最终目标是让 Hero Radar 越来越接近用户自己的项目判断标准。